

Une formalisation idiomatique en Mathlib de l'involution de Fricke et de son opérateur sur les formes modulaires

Xavier Callens*

SocrateAI Research Initiative

6 septembre 2026

Archivé sur Zenodo : [10.5281/zenodo.22541930](https://zenodo.org/record/22541930)

Résumé

Nous formalisons, en Lean 4 au-dessus de Mathlib, l'involution de Fricke $W_N = \begin{pmatrix} 0 & -1 \\ N & 0 \end{pmatrix}$ sur $\Gamma_0(N)$ ainsi que l'opérateur qu'elle induit sur les formes modulaires, sous la forme d'une petite API réutilisable plutôt que d'une infrastructure interne à une démonstration. **Nous ne revendiquons aucune antériorité** : du matériel relatif à Fricke et à Atkin–Lehner existe déjà en Lean 4 dans l'artefact du dernier théorème de Fermat [8] (§6), qui couvre strictement davantage. Ce que nous proposons est un conditionnement destiné à une intégration en amont : Mathlib elle-même ne contient aucun opérateur de Fricke ni d'Atkin–Lehner. Mathlib fournit déjà le demi-plan de Poincaré, l'action de Möbius de $GL_2(\mathbb{R})$, l'action « slash » en poids k portant le facteur déterminant, les sous-groupes de congruence $\Gamma_0(N)$, ainsi que les types structurés `SlashInvariantForm` et `ModularForm` ; elle ne contient pas l'opérateur. Nous fournissons cette pièce manquante en trois couches : (i) au niveau matriciel — $W_N \in GL_2^+(\mathbb{R})$ avec $\det W_N = N$, l'identité d'involution $W_N^2 = -N \cdot I$, la conjugaison explicite $W_N \gamma W_N^{-1} = \begin{pmatrix} d & -c/N \\ -Nb & a \end{pmatrix}$, et le théorème selon lequel W_N normalise $\Gamma_0(N)$; (ii) au niveau « slash » — $f \mapsto f|_k W_N$ préserve l'invariance slash sous $\Gamma_0(N)$; et (iii) au niveau structuré — l'opérateur est bien défini sur `ModularForm`, son holomorphie et son caractère borné aux pointes étant transportés le long de W_N . Le développement compte 27 déclarations en 291 lignes ; les 21 théorèmes compilent sans aucun `sorry` et dépendent exactement des trois axiomes standard de Lean, `propext`, `Classical.choice` et `Quot.sound`, épinglés par 16 contrôles `#guard_msgs` bloquant la compilation, de sorte qu'une dérive de l'empreinte axiomatique provoque une erreur de compilation. Nous ne revendiquons aucune mathématique nouvelle : les résultats sont classiques, dus à Fricke et à Atkin–Lehner [1]. L'apport est la mécanisation, et son usage visé est de servir d'infrastructure à la formalisation des identités de quotients éta, où les valeurs propres de Fricke constituent l'invariant organisateur.

MSC 2020 : 11F11, 11F06, 68V20. **Mots-clés** : involution de Fricke, théorie d'Atkin–Lehner, sous-groupes de congruence, Lean 4, Mathlib, vérification formelle.

1 Introduction

L'involution de Fricke s'énonce de façon élémentaire et intervient partout dans la théorie des formes modulaires sur $\Gamma_0(N)$. Pour $N \geq 1$, posons

$$W_N = \begin{pmatrix} 0 & -1 \\ N & 0 \end{pmatrix}, \quad \det W_N = N > 0.$$

*Auteur correspondant : callensxavier@gmail.com

Agissant sur le demi-plan de Poincaré par $\tau \mapsto -1/(N\tau)$, elle échange les pointes 0 et ∞ de $\Gamma_0(N)$, normalise $\Gamma_0(N)$, et induit une involution de $M_k(\Gamma_0(N))$ dont les sous-espaces propres ± 1 organisent une large part de l’arithmétique de l’espace — de la manière la plus visible pour les quotients éta, où la multiplicativité se lit sur les valeurs propres de Fricke et d’Atkin–Lehner [7, 6].

Cette note est écrite dans le mode que la pratique récente rend standard : un artefact formel livré conjointement à l’exposition destinée aux lecteurs humains. L’autoformalisation à grande échelle a franchi un seuil — une démonstration complète et vérifiée par machine du dernier théorème de Fermat a été produite en onze jours par des agents d’intelligence artificielle coordonnés par un graphe d’énoncés partagé [8, 9] — et les questions méthodologiques afférentes relèvent désormais des mathématiques ordinaires [10, 3]. Nous travaillons en Lean 4 [12] au-dessus de Mathlib [2], et nous adoptons deux pratiques précises issues de l’artefact FLT : l’audit axiomatique en tant que *cible de compilation*, et un graphe explicite de dépendances entre énoncés (§5).

Ce que Mathlib fournit déjà

Nous sommes explicites sur l’état de l’art, car il détermine ce qui est nouveau ici et ce qui ne l’est pas. Au commit **905b9581** (28 juillet 2026), Mathlib [2] contient, sous des formes strictement plus générales que ce qu’il nous faudrait réénoncer :

- `UpperHalfPlane` et l’action de Möbius de $\text{GL}(\text{Fin } 2) \mathbb{R}$, ainsi que `denom_ne_zero_of_im` et `normSq_denom_pos` ;
- `im_smul_eq_div_normSq`, qui est exactement l’identité classique $\text{Im}(M \cdot \tau) = |\det M| \text{Im}(\tau) / |c\tau + d|^2$, énoncée pour $M \in \text{GL}_2(\mathbb{R})$ arbitraire via $|\det M|$ et non seulement pour $\det M > 0$;
- `Matrix.GLPoS`, le sous-groupe des matrices de déterminant positif ;
- `CongruenceSubgroup.Gamma0 N` ;
- l’action « slash » en poids k , `SlashAction ℤ (GL (Fin 2) ℝ) (ℍ → ℂ)`, dont la définition porte déjà le facteur déterminant : $f|_k g = \sigma(g) f(g \cdot \tau) |\det g|^{k-1} \text{denom}(g, \tau)^{-k}$;
- le type structuré `ModularForm Γ k` pour $\Gamma : \text{Subgroup}(\text{GL}(\text{Fin } 2) \mathbb{R})$, avec holomorphie et caractère borné aux pointes.

Rien de tout cela n’est redémontré ici ; nous l’importons. Un lecteur cherchant une formalisation de l’action de $\text{GL}_2^+(\mathbb{R})$ sur $\mathbb{H}$ doit utiliser celle de Mathlib, non la nôtre.

La lacune, et notre apport

Une recherche sur l’ensemble du dépôt Mathlib à ce commit pour `Fricke` ou `AtkinLehner` ne renvoie *aucune* occurrence : la machinerie permettant d’héberger ces opérateurs existe dans Mathlib, mais les opérateurs n’y sont pas. C’est la lacune que nous comblons — avec la réserve importante, développée au §6, que du matériel relatif à Fricke *existe bel et bien* en Lean 4 hors de Mathlib. Nos apports sont donc énoncés comme une API, et non comme des premières :

- (C1) W_N comme élément de $\text{GL}(\text{Fin } 2) \mathbb{R}$ avec $\det W_N = N$, et $W_N \in \text{GL}_2^+(\mathbb{R})$ (théorème 3.1) ;
- (C2) $W_N^2 = -N \cdot I$ (théorème 3.2), ce qui, les matrices scalaires agissant trivialement sur $\mathbb{H}$, constitue la propriété d’involution ;
- (C3) la formule de conjugaison explicite (théorème 3.3) ;

- (C4) W_N conjugue $\Gamma_0(N)$ dans lui-même à l'intérieur de $GL_2(\mathbb{R})$ (théorème 3.4); joint au fait que $W_N^2 = -N \cdot I$ est central, cela donne la condition de normalisateur bilatère, mais c'est l'inclusion que nous formalisons;
- (C5) le slash par W_N préserve l'invariance slash sous $\Gamma_0(N)$, d'où l'opérateur sur `SlashInvariantForm` (théorème 4.1);
- (C6) l'holomorphie et le caractère borné aux pointes se transportent le long de W_N , d'où l'opérateur sur le type structuré `ModularForm` (théorème 4.2);
- (C7) le composé $f \mapsto (f|_k W_N)|_k W_N$ est la multiplication par le scalaire explicite $(-1)^k N^{k-2}$ (théorème 4.3), ce qui identifie la normalisation faisant de $|_k W_N$ une involution.

(C4) est le point charnière : c'est ce qui rend $f \mapsto f|_k W_N$ bien défini. (C5) et (C6) le remontent jusqu'aux types structurés de Mathlib, afin que l'opérateur de Fricke puisse se composer avec le reste de la bibliothèque plutôt que de vivre à côté d'elle.

2 Rappels mathématiques

Soit $\gamma = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in SL_2(\mathbb{Z})$ avec $\gamma \in \Gamma_0(N)$, c'est-à-dire $N \mid c$; écrivons $c = Nc'$. Un calcul direct donne

$$W_N \gamma W_N^{-1} = \begin{pmatrix} d & -c' \\ -Nb & a \end{pmatrix}, \quad (1)$$

dont les coefficients sont entiers précisément parce que $N \mid c$, dont le déterminant vaut $da - (-c')(-Nb) = ad - bc = 1$, et dont le coefficient inférieur gauche $-Nb$ est divisible par N . Le conjugué appartient donc de nouveau à $\Gamma_0(N)$, et W_N normalise $\Gamma_0(N)$.

Plutôt que de manipuler W_N^{-1} dans la formalisation, nous démontrons l'identité *d'entrelacement* équivalente

$$W_N \cdot \gamma = \delta \cdot W_N, \quad \delta := \begin{pmatrix} d & -c' \\ -Nb & a \end{pmatrix}, \quad (2)$$

qui évite entièrement les inverses et dont (1) découle par une multiplication à droite. Les deux membres de (2) valent $\begin{pmatrix} -c & -d \\ Na & Nb \end{pmatrix}$, l'accord ayant lieu exactement sous la relation $c = Nc'$.

3 Le développement Lean

Le développement comporte quatre modules. Nous reproduisons les déclarations porteuses verbatim depuis les sources compilées.

```
def frickeMatrix (N : ℕ) : Matrix (Fin 2) (Fin 2) ℝ := !![0, -1; (N : ℝ), 0]

@[simp] theorem frickeMatrix_det (N : ℕ) : (frickeMatrix N).det = (N : ℝ) := by
  simp [frickeMatrix, Matrix.det_fin_two_of]

noncomputable def frickeW {N : ℕ} (hN : 0 < N) : GL (Fin 2) ℝ :=
  Matrix.GeneralLinearGroup.mkOfDetNeZero (frickeMatrix N) (frickeMatrix_det_ne_zero hN)
```

Listing 1 : La matrice de Fricke et son déterminant.

Théorème 3.1 (W_N est de déterminant positif). *frickeW_mem_GLPos* : *frickeW hN* ∈ *GLPos* (*Fin 2*) ℝ.

Théorème 3.2 (Involution). $W_N^2 = -N \cdot I : \text{frickeW_sq_coe} : ((\text{frickeW } hN * \text{frickeW } hN : GL (Fin 2) \mathbb{R}) : Matrix _ _ \mathbb{R}) = (-(N : \mathbb{R})) \bullet 1$.

Les matrices scalaires agissant trivialement sur $\mathbb{H}$, le théorème 3.2 exprime précisément que W_N induit une involution du demi-plan de Poincaré.

L'ingrédient arithmétique est la divisibilité extraite de l'appartenance à $\Gamma_0(N)$, que Mathlib énonce modulo N :

```
theorem gamma0_dvd_lower_left {N : ℕ} {γ : SL(2, ℤ)} (hγ : γ ∈ Gamma0 N) :
  (N : ℤ) ∣ γ.1 1 0 := by
  rw [Gamma0_mem] at hγ
  exact (ZMod.intCast_zmod_eq_zero_iff_dvd _ _).mp hγ
```

Listing 2 : Divisibilité du coefficient inférieur gauche.

Le conjugué δ de (1) se définit alors comme un véritable élément de $SL_2(\mathbb{Z})$, sa condition de déterminant étant acquittée par `linear_combination` contre $\det \gamma = 1$:

```
def frickeConj {N : ℕ} (γ : SL(2, ℤ)) (c' : ℤ) (hc : γ.1 1 0 = (N : ℤ) * c') : SL(2, ℤ) :=
  (!![γ.1 1 1, -c'; -(N : ℤ) * γ.1 0 1, γ.1 0 0], by
    have hdet := γ.2
    rw [Matrix.det_fin_two] at hdet; rw [hc] at hdet
    rw [Matrix.det_fin_two_of]; linear_combination hdet)
```

Listing 3 : Le conjugué de Fricke, avec sa preuve de déterminant.

Théorème 3.3 (Conjugaison). $\text{frickeW_conj_eq} : \text{pour } \gamma \in SL_2(\mathbb{Z}) \text{ avec } c = Nc',$

$$W_N \cdot \text{mapGL}_{\mathbb{R}}(\gamma) \cdot W_N^{-1} = \text{mapGL}_{\mathbb{R}}(\text{frickeConj } \gamma c').$$

La démonstration réécrit par l'identité d'entrelacement (2) et simplifie $W_N W_N^{-1}$.

Théorème 3.4 (Résultat principal : normalisation). $\text{frickeW_normalizes_Gamma0} :$

$$\forall g \in \text{Gamma0}(N). \text{map}(\text{mapGL}_{\mathbb{R}}), \quad W_N g W_N^{-1} \in \text{Gamma0}(N). \text{map}(\text{mapGL}_{\mathbb{R}}).$$

Démonstration (telle que formalisée). Décomposer $g = \text{mapGL}_{\mathbb{R}}(\gamma)$ avec $\gamma \in \Gamma_0(N)$; obtenir c' tel que $c = Nc'$ par `gamma0_dvd_lower_left`; réécrire par le théorème 3.3; conclure par `Subgroup.mem_map_of_mem` appliqué à `frickeConj_mem_Gamma0`, dont le contenu est que le coefficient inférieur gauche $-Nb$ s'annule modulo N . $\square$

4 L'opérateur sur les formes modulaires

La normalisation est un énoncé sur les groupes; être un *opérateur sur les formes modulaires* exige de transporter le long de W_N les deux conditions analytiques de `ModularForm`. Nous procédons en deux étapes, en travaillant avec l'image $\Gamma_0(N)_{\mathbb{R}} := \Gamma_0(N). \text{map}(\text{mapGL}_{\mathbb{R}})$ dans $GL_2(\mathbb{R})$, qui est le sous-groupe sur lequel les types structurés de Mathlib sont énoncés.

Théorème 4.1 (Invariance slash). $\text{slash_frickeW_invariant} : \text{pour } f \text{ une } \text{SlashInvariantForm} \text{ de poids } k \text{ pour } \Gamma_0(N)_{\mathbb{R}} \text{ et tout } g \text{ dans ce sous-groupe, } (f|_k W_N)|_k g = f|_k W_N.$

Démonstration (telle que formalisée). Écrire $g = \text{mapGL}_{\mathbb{R}}(\gamma)$ avec $\gamma \in \Gamma_0(N)$ et extraire $c = Nc'$. Alors, en utilisant le cocycle du slash dans les deux sens et l'identité d'entrelacement (2),

$$(f|_k W_N)|_k \gamma = f|_k(W_N \gamma) = f|_k(\delta W_N) = (f|_k \delta)|_k W_N = f|_k W_N,$$

la dernière étape par invariance slash de f sous $\delta \in \Gamma_0(N)$ (théorème 3.3). Cela se conditionne en `frickeSlashOperator`. $\square$

Théorème 4.2 (Opérateur sur `ModularForm`). *`frickeModularOperator` : $f \mapsto f|_k W_N$ est une application `ModularForm` $\Gamma_0(N)_{\mathbb{R}} \rightarrow \text{ModularForm } \Gamma_0(N)_{\mathbb{R}} \rightarrow \text{ModularForm } \Gamma_0(N)_{\mathbb{R}} \rightarrow \text{ModularForm } \Gamma_0(N)_{\mathbb{R}}$.*

Démonstration (telle que formalisée). L'holomorphie est immédiate à partir de `MDifferentiable.slash` de Mathlib : le slash par tout élément fixé de $GL_2(\mathbb{R})$ préserve `MDiff`. Pour le caractère borné, `OnePoint.IsBoundedAt.smul_iff` de Mathlib réduit $f|_k W_N$ est borné en c à f est borné en $W_N \cdot c$, de sorte qu'il suffit que W_N envoie les pointes de $\Gamma_0(N)$ sur des pointes de $\Gamma_0(N)$. C'est `isCusp.frickeW.smul`, obtenu à partir de `isCusp.smul` de Mathlib, de la forme « sous-groupe » du théorème 3.4 (`frickeW_conjAct_le`) et de la monotonie de `isCusp` en son argument de sous-groupe. $\square$

Concrètement, W_N échange les pointes 0 et ∞ de $\Gamma_0(N)$; ce qui est démontré ici est l'énoncé plus faible, suffisant pour le transport, que l'ensemble des pointes est préservé.

Théorème 4.3 (Le composé est un scalaire). *`frickeW_sq_slash` : pour toute $f : \mathbb{H} \rightarrow \mathbb{C}$ et tout poids k ,*

$$(f|_k W_N)|_k W_N = (N^2)^{k-1}(-N)^{-k} f = (-1)^k N^{k-2} f.$$

Démonstration (telle que formalisée). Par la loi de cocycle, le membre de gauche vaut $f|_k(W_N^2)$, et $W_N^2 = -N \cdot I$ est scalaire (théorème 3.2). Pour cette matrice, les trois ingrédients du slash de Mathlib se calculent directement : $\det(W_N^2) = N^2$, $\text{denom}(W_N^2, \tau) = -N$, et W_N^2 agit trivialement sur $\mathbb{H}$. Le facteur de conjugaison σ ne nécessite aucune disjonction de cas sur le déterminant : $\sigma(gg') = \sigma g \circ \sigma g'$ et $\sigma g \circ \sigma g = \text{id}$ donnent $\sigma(W_N^2) = \text{id}$ immédiatement. $\square$

Pour k pair, il s'ensuit que $f \mapsto N^{1-k/2}(f|_k W_N)$ est de carré l'identité — c'est l'involution de Fricke proprement dite. Pour k impair la question est vide, puisque $-I \in \Gamma_0(N)$ force $f \equiv 0$ (`ModularForm.eq_zero_of_neg_one_mem` de Mathlib).

5 État de la vérification

Toutes les déclarations du module compilent. Il n'y a aucun `sorry` et aucun axiome propre au projet. L'empreinte axiomatique de chaque théorème, obtenue par `#print axioms`, est exactement

[`propext`, `Classical.choice`, `Quot.sound`],

les trois axiomes standard de la logique de Lean. À la suite de l'artefact du dernier théorème de Fermat [8], l'empreinte n'est pas seulement rapportée mais *imposée* : un module `FinalCheck` place une garde `#guard_msgs in #print axioms` sur chacun des seize théorèmes principaux à l'intérieur de la cible de compilation par défaut, de sorte que tout élargissement de l'ensemble d'axiomes est une erreur de compilation ; un contrôle négatif (une garde affirmant une empreinte délibérément fausse) est conservé hors de la compilation et il a été vérifié qu'il échoue. Le développement est suivi comme un graphe de dépendances entre énoncés (`dag/theorems.jsonl`, avec une carte lisible dans `dag/PROOF-PATH.md`) ; la machine vérifie que tout nœud marqué démontré nomme une déclaration existante et ne dépend que de nœuds démontrés. Nous ne revendiquons rien au-delà de ce que le noyau vérifie : dans la terminologie adoptée dans l'ensemble du programme SocrateAI, (C1)–(C7) sont des résultats *formels* ; l'exposition qui les entoure ne l'est pas.

Artefact. Archivé sur Zenodo, [doi:10.5281/zenodo.22541930](https://doi.org/10.5281/zenodo.22541930), qui constitue la version de référence citable de cette note et de ses sources. Dépôt <https://github.com/xaviercallens/SocrateAI-Lean-Lib> (étiquette `fricke-v1`) ; modules `Lean/SocrateAI/ModularForms/{FrickeInvolution, FrickeSlash, FrickeModular, FrickeComposite}.lean`, les gardes axiomatiques se trouvant dans `Lean/SocrateAI/FinalCheck.lean` ; un miroir accompagné d’une fiche descriptive est disponible à <https://huggingface.co/datasets/callensxavier/socrateai-fricke-involution>. Chaîne d’outils `leanprover/lean4:v4.32.2` ; Mathlib figée au commit `905b9581` (28 juillet 2026) ; `lake build SocrateAI` s’achève en 3053 tâches (module de gardes axiomatiques inclus). Les quatre modules totalisent 27 déclarations en 291 lignes et n’importent que `Mathlib.NumberTheory.ModularForm`, `Mathlib.NumberTheory.ModularForms.CongruenceSubgroups`, `Mathlib.NumberTheory.ModularForms.SlashInvolution` et `Mathlib.NumberTheory.ModularForms.Basic`. La configuration de compilation n’est *pas* distribuée : notre `lakefile.lean` code en dur des chemins absolus vers un dépôt local de paquets Mathlib, de sorte que l’artefact n’est pas encore compilable par un tiers sans qu’il fournisse sa propre copie de Mathlib au commit indiqué. Un `lakefile` portable reste à écrire, et nous le signalons comme l’obstacle principal à une réplcation indépendante.

6 Travaux apparentés

Formalisation antérieure en Lean 4 de matériel relatif à Fricke. L’artefact du dernier théorème de Fermat [8] est un développement Lean 4 construit sur Mathlib qui contient déjà une théorie substantielle de Fricke et d’Atkin–Lehner. Une recherche de code à l’échelle du dépôt renvoie de l’ordre de $2,7 \times 10^3$ fichiers Lean mentionnant `Fricke` et $3,6 \times 10^3$ mentionnant `AtkinLehner`, dont `Thm_ModularForm_heckeU_add_slash_fricke_eq_zero`, `Thm_CuspForm_hasNebentypus_involution` et `Def_ModularCurve_AtkinLehner`. Leurs énoncés utilisent la même matrice $\begin{pmatrix} 0 & -1 \\ N & 0 \end{pmatrix}$ et la même action slash de Mathlib que nous employons ici, et ils vont considérablement plus loin — jusqu’aux opérateurs de Hecke, aux formes propres à nebentypus, et aux opérateurs d’Atkin–Lehner sur les courbes modulaires.

De la façon la plus directe, `Def_CuspForm_AtkinLehnerOperator.lean` définit `atkinLehnerLin`, une application `ModularForm (Γ0(M)) k →ℓ[C] ModularForm (Γ0(M)) k`, acquittant l’invariance slash, l’holomorphie et le caractère borné aux pointes comme le fait notre `frickeModularOperator` — soit nos (C5) et (C6), pour toute la famille d’Atkin–Lehner et sous forme d’application $\mathbb{C}$ -linéaire. Leur `Def_ModularForm_AtkinLehnerDatum.lean` construit son élément de groupe avec le même idiome `mkOfDetNeZero` que nous et démontre l’analogue de nos (C1), (C2) et (C7).

En particulier, leur `CohCarrier.frickeMat` est, à la dénomination près, notre `frickeConj` : la même matrice conjuguée $\begin{pmatrix} d & -c/N \\ -Nb & a \end{pmatrix}$, la même hypothèse de divisibilité, et une preuve de déterminant qui s’achève elle aussi par `linear_combination`. Notre développement a été obtenu indépendamment, ce qui n’a aucune incidence sur l’antériorité ; nous consignons la coïncidence parce qu’un lecteur comparant les deux doit savoir qu’il s’agit de la même construction.

Ce qui est néanmoins proposé ici. Le matériel FLT est interne à une démonstration : il vit dans un espace de noms `CohCarrier` ad hoc, et W_N y est généralement passé à chaque théorème comme un paramètre contraint par une équation matricielle. Nous définissons au contraire W_N une fois pour toutes comme un élément de $GL_2(\mathbb{R})$ et démontrons la petite théorie environnante — appartenance à GL_2^+ , $W_N^2 = -N \cdot I$, la formule de conjugaison, et les opérateurs induits sur `SlashInvariantForm` et `ModularForm`. La destination visée est Mathlib, qui ne contient rien de tout cela. Énoncé précisément, ce qui subsiste après divulgation est : la matrice littérale W_N sur $\mathbb{R}$ (le coefficient inférieur droit de leur donnée est q , si bien qu’elle ne se spécialise pas exactement en W_N), le scalaire composé explicite de (C7), et un développement à quatre importations nommées utilisant les tactiques standard plutôt qu’un harnais de preuve dédié. C’est une différence de méthode, et savoir si elle vaut quelque chose est une question pour la relecture Mathlib, non une chose qu’il nous appartient d’affirmer.

7 Limites

Les opérateurs de Fricke et d’Atkin–Lehner sont classiques ; le traitement de référence des involutions d’Atkin–Lehner W_Q pour $Q \parallel N$ est [1], et des exposés de manuel figurent dans [4, 5, 6]. Notre apport n’est pas une nouveauté mathématique mais le transfert de ce matériel dans un cadre vérifié par machine où l’on peut s’appuyer dessus.

Nous énonçons les limites sans détour.

Remarque 7.1 (Ce qui n’est *pas* fait ici).

- (a) Nous ne formalisons que l’involution de Fricke W_N . La famille complète d’Atkin–Lehner W_Q pour $Q \parallel N$, et la décomposition simultanée en sous-espaces propres qui en résulte, ne sont pas traitées.
- (b) La décomposition de $M_k(\Gamma_0(N))$ en sous-espaces propres ± 1 n’est pas elle-même conditionnée comme une définition Lean. Le contenu mathématique est le théorème 4.3, qui est formalisé ; ce qui reste relève de la tenue de comptes — porter l’hypothèse « k pair » à travers l’arithmétique `zpow` sur la constante, et emballer les deux sous-espaces propres comme sous-modules.
- (c) Nous ne faisons aucune revendication d’antériorité. Notre vérification de lacune a été menée contre Mathlib, où elle est valide ; une recherche ultérieure dans les développements Lean 4 cités ici a mis au jour l’état de l’art décrit au §6. Nous n’avons exploré ni le Zulip Lean ni les autres assistants de preuve.

8 Application visée

La motivation est en aval. Les quotients $\hat{\eta}$ $f(\tau) = \prod_{\delta \mid N} \eta(\delta\tau)^{r_\delta}$ sont modulaires sur $\Gamma_0(N)$ sous les conditions de Ligozat, et leur comportement sous les opérateurs de Fricke et d’Atkin–Lehner est ce qui rend la multiplicativité traitable [7]. Une formalisation des identités de quotients $\hat{\eta}$ requiert donc W_N comme prérequis, et c’est ce que ce module fournit. Le même opérateur est l’entrée du côté modulaire des correspondances de « moonshine » étudiées ailleurs dans ce programme. L’opérateur étant désormais défini sur `ModularForm`, les étapes suivantes immédiates sont la constante de normalisation de la limite (b) et la famille complète d’Atkin–Lehner de la limite (a). Du côté analytique, les bornes de coefficients de type Rademacher qui accompagnent le programme des quotients $\hat{\eta}$ seront d’abord vérifiées au niveau « validé par programme » à l’aide d’un outillage léger de preuve d’inégalités dans l’esprit de l’assistant *estimates* de Tao [11], et seulement ensuite promues au rang d’énoncés vérifiés par le noyau — la même discipline à deux niveaux (vérificateur exact en dessous, noyau au-dessus) employée dans tout ce programme.

9 Conclusion

Nous avons formalisé en Lean 4 l’involution de Fricke W_N sur $\Gamma_0(N)$ et l’opérateur qu’elle induit sur les formes modulaires, comblant une lacune de *Mathlib* — mais non, comme le consigne le §6, de Lean 4 dans son ensemble. Le développement est petit — 27 déclarations en 291 lignes —, ne dépend que des trois axiomes standard de Lean, et est bâti sur l’infrastructure `SlashAction`, `SlashInvariantForm` et `ModularForm` existante de Mathlib plutôt que de la dupliquer, de sorte qu’il se compose avec le reste de la bibliothèque. Les mathématiques sont classiques ; leur énoncé vérifié par machine ne l’est pas.

Note sur cette version

Le présent texte est la version française d’une note rédigée initialement en anglais et déposée sur Zenodo sous le DOI [10.5281/zenodo.22541930](https://doi.org/10.5281/zenodo.22541930). Les deux versions ont le même contenu mathématique ; en cas de divergence, la version anglaise fait foi pour les énoncés formels, dont les noms Lean ne sont pas traduits. Cette note n’a pas fait l’objet d’une relecture par les pairs : notre propre relecture interne a conclu qu’elle ne constituait pas une soumission de revue, et la voie recommandée pour ce matériel est une contribution (*pull request*) à Mathlib. Le dépôt HAL vaut archivage et citation, non validation.

Références

- [1] A. O. L. Atkin, J. Lehner, *Hecke operators on $\Gamma_0(m)$* , Math. Ann. **185** (1970), 134–160. doi:[10.1007/BF01359701](https://doi.org/10.1007/BF01359701).
- [2] The mathlib Community, *The Lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP), 2020.
- [3] K. Buzzard, *Formalising mathematics*, Math. Proc. Cambridge Philos. Soc., 2021.
- [4] F. Diamond, J. Shurman, *A First Course in Modular Forms*, Graduate Texts in Mathematics **228**, Springer (2005).
- [5] T. Miyake, *Modular Forms*, Springer Monographs in Mathematics (2006).
- [6] K. Ono, *The Web of Modularity : Arithmetic of the Coefficients of Modular Forms and q -series*, CBMS Regional Conference Series in Mathematics **102**, AMS (2004). doi:[10.1090/cbms/102](https://doi.org/10.1090/cbms/102).
- [7] Y. Martin, *Multiplicative eta-quotients*, Trans. Amer. Math. Soc. **348** (1996), 4825–4856.
- [8] Anthropic, *A formal proof of Fermat’s Last Theorem in Lean 4*, artefact logiciel, <https://github.com/anthropics/fermats-last-theorem>, 2026.
- [9] S. Chen, K. Marwaha, X. Lu, H. Yuen, T. Peng, *Prove2Me : An open collaborative platform for scaling math formalization*, arXiv :2608.28433, 2026.
- [10] T. Tao, *Mathematics in the age of AI*, conférence au Congrès international des mathématiciens 2026 ; arXiv :2608.16753.
- [11] T. Tao, *estimates : a lightweight proof assistant for inequalities and asymptotic estimates*, logiciel, <https://github.com/teorth/estimates>, 2025.
- [12] L. de Moura et al., *The Lean 4 Theorem Prover and Programming Language*, CADE-28, LNCS **12699**, Springer (2021).